BÖLÜM 5 / MODULE 5

Asenkron İletişim, Mesajlaşma ve Olay Tabanlı Mimari

Asynchronous Messaging & Event-Driven Architecture

Zamansal ayrışma, RabbitMQ vs Apache Kafka, Kafka partition ve consumer grupları, Exactly-Once garantileri, Idempotency ve Transactional Outbox deseni.

Temporal decoupling, RabbitMQ vs Apache Kafka, Kafka commit logs & consumer groups, Exactly-Once semantics, Idempotency tokens, and the Transactional Outbox pattern.

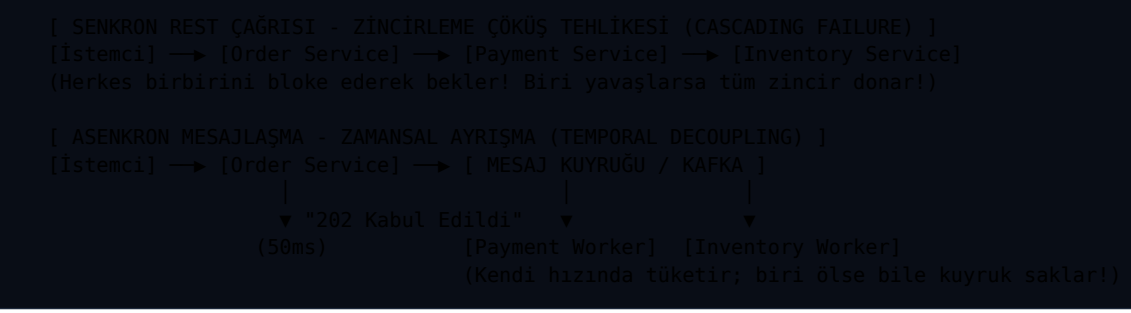
25

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Senkron vs Asenkron İletişim: Zamansal Ayrışma (Temporal Decoupling)

Synchronous vs Asynchronous: Temporal Decoupling & Buffering

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Senkron iletişim, arkadaşınızın evine gidip kapısını çalmak ve kapıyı açana kadar karda kışta 2 saat kapının önünde dikilmektir; arkadaşınız evde yoksa işiniz mahvolur! Asenkron iletişim ise hediyeyi arkadaşınızın posta kutusuna (Mesaj Kuyruğu) bırakıp yolunuza devam etmektir. Arkadaşınız isterse 3 saat sonra, isterse ertesi gün eve gelsin; mektubu kutudan alır ve okur. Sistemlerinizin birbirinin ayakta olmasına bağımlı olmadan çalışması (Zamansal Ayrışma) işte bu posta kutusu sayesindedir!

🇬🇧 ENGLISH (ELI5)

Synchronous communication is ringing your friend's doorbell and standing frozen in a blizzard for 2 hours waiting for them to open up; if they aren't home, you freeze to death! Asynchronous communication is dropping a parcel into their locked mailbox (Message Queue) and going about your business. Whether your friend gets home in 10 minutes or tomorrow morning, the parcel sits safe in the mailbox. Decoupling services in time (Temporal Decoupling) ensures no service halts waiting for another!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Senkron (HTTP REST, gRPC) iletişimde çağıran servis yanıt gelene kadar bir işletim sistemi iş parçacığını (thread) ve soketini bloke eder. Dağıtık mimarilerde A -> B -> C -> D şeklindeki senkron çağrı zincirleri 'Cascading Failure' (zincirleme çöküş) yaratır; D servisi 5 saniye gecikirse A'nın tüm thread havuzu tükenir. Asenkron iletişim (Message Queues) zamansal ayrışma (Temporal Decoupling) sağlar: 1) İstemci isteği kuyruğa atıp anında 'HTTP 202 Accepted' alır. 2) Kuyruk bir tampon (Shock Absorber) görevi görerek ani trafik patlamalarını emer. 3) Tüketici servisler çökseler bile yeniden başladıklarında kuyrukta biriken mesajları kaldıkları yerden işlemeye devam eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Synchronous invocations (REST, gRPC) hold operating system threads and network sockets hostage until upstream downstream services respond. In deep call chains (A -> B -> C -> D), a latency spike in service D exhausts thread pools across all upstream hosts, triggering catastrophic cascading outages. Asynchronous messaging decouples producers and consumers temporally: 1) Producers publish to durable message brokers and return HTTP 202 immediately. 2) Queues act as shock absorbers buffering flash traffic spikes. 3) Downstream worker failure causes zero data loss; consumers catch up monotonically upon restart.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Asenkron İş Kuyruğu ve İşçi Modeli (Virtual Threads Destekli)
import java.util.concurrent.*;

public class AsyncJobQueue {
    public record OrderJob(String orderId, double amount, long timestamp) {}

    private final BlockingQueue<OrderJob> queue = new LinkedBlockingQueue<>(1000);

    public boolean publishJob(OrderJob job) {
        return queue.offer(job); // Kuyruk dolmadığı sürece bloke etmeden at
    }

    public void startWorker() {
        Thread.ofVirtual().start(() -> {
            while (true) {
                try {
                    OrderJob job = queue.take(); // Mesaj gelene kadar bekle
                    IO.println("İşlendi: Sipariş #" + job.orderId() + ", Tutar: $" + job.amount());
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        });
    }
}

void main() throws InterruptedException {
    var jobQueue = new AsyncJobQueue();
    jobQueue.startWorker();

    jobQueue.publishJob(new AsyncJobQueue.OrderJob("ORD_991", 149.99, System.currentTimeMillis()));
    Thread.sleep(50); // Sanal iş parçacığının işlemesini bekle
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Asenkron mimarinin bedeli 'Eventual Consistency' (Nihai Tutarlılık) ve karmaşık hata yönetimidir. Kullanıcıya 'Siparişiniz alındı, hazırlanıyor' mesajı vermeniz gerekir; kullanıcı anında faturasını göremeyebilir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

The trade-off of asynchronous messaging is eventual consistency and complex failure compensation. The UI must adapt to an asynchronous workflow ('Your order is processing') rather than assuming instantaneous transactional guarantees.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Asenkron iletişim servisler arasındaki zamansal ve mekansal bağımlılığı koparır.
- Mesaj aracıları ani trafik dalgalanmalarında doğal bir amortisör (shock absorber) görevi görür.
- Senkron çağrı zincirleri yerine asenkron olaylar kullanmak zincirleme çöküşleri engeller.

🎯 Key Takeaways for the Interview

- Asynchronous communication eliminates temporal and spatial coupling across services.
- Message queues act as elastic buffers absorbing sudden traffic spikes without dropping requests.
- Replacing synchronous call chains with asynchronous queues prevents cascading system outages.

26

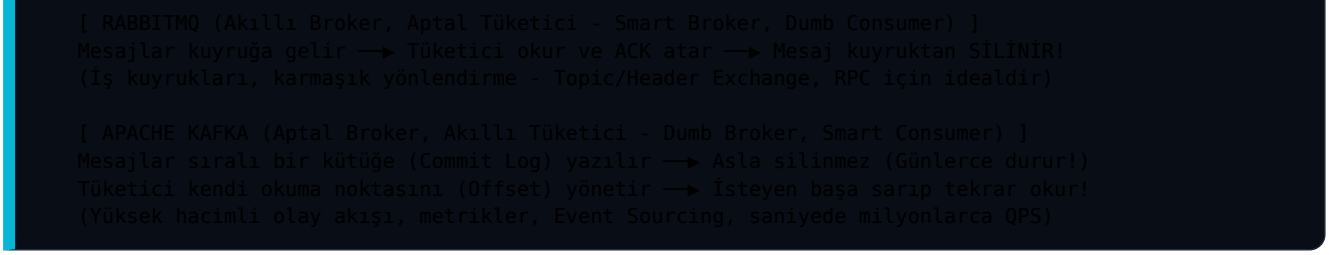
BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Mesaj Kuyrukları vs Olay Akışları: RabbitMQ vs Apache Kafka

Message Queues vs Event Streams: RabbitMQ vs Apache Kafka

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

RabbitMQ bir postane kuryesidir: Çantasından mektubu çıkarır, kapınızı çalar, elinize verir ve imzayı aldıktan sonra (ACK) o mektubu sistemden tamamen siler. Mektup bir daha geri gelmez. Apache Kafka ise gazete arşividir: Yazılan tüm haberler devasa ciltli bir deftere (Append-Only Commit Log) sırayla yapıştırılır ve ASLA yırtılıp atılmaz! Siz bir araştırmacı olarak defterin 50. sayfasına kadar okuyabilirsiniz (Offset); arkadaşınız ise 10. sayfada olabilir. Hatta isterseniz 1. sayfaya geri dönüp tüm yılı baştan sona tekrar okuyabilirsiniz (Replayability)!

🇬🇧 ENGLISH (ELI5)

RabbitMQ is a specialized courier: It pulls a letter from its satchel, rings your doorbell, hands it to you, collects your signature (ACK), and deletes the letter from the system forever. Apache Kafka is a bound newspaper archive: every article published is permanently glued onto an immutable, sequential commit log and NEVER discarded! You can read up to page 50 (Offset), while a colleague reads at page 10. You can even rewind your bookmark back to page 1 and reread an entire decade of history from scratch (Replayability)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) RabbitMQ (AMQP): Mesaj odaklıdır. Akıllı yönlendirme (Exchange tipleri: Direct, Fanout, Topic, Headers). Mesaj tüketici tarafından onaylandığında (ACK) broker'dan temizlenir. Düşük gecikmeli, karmaşık kurallı iş kuyrukları ve bildirimler için mükemmeldir; ancak geriye dönük yeniden okuma (Replay) desteklemez. 2) Apache Kafka: Dağıtık, append-only commit-log mimarisidir. Broker mesajı silmez; mesajlar retention süresi (ör. 7 gün) boyunca diskte sıralı tutulur. Tüketiciler kendi okuma indekslerini (`Offset`) yönetir. Sıfır Kopyalama (Zero-Copy OS sendfile) tekniği sayesinde saniyede milyonlarca olayı (GB/sn bant genişliği) kayıpsız işler. Event Sourcing, CQRS ve analitik akışlar için MAANG standardıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) RabbitMQ (AMQP): Message-oriented broker with sophisticated routing topologies (Exchange bindings: Direct, Topic, Fanout, Headers). Once acknowledged, messages are expunged from queue memory. Tailored for discrete background jobs, task queues, and low-latency notifications; lacks historical replayability. 2) Apache Kafka: Distributed, append-only, partitioned commit log. Messages persist immutably regardless of consumer state according to retention policies (e.g. 7 days or infinite compaction). Consumers advance localized offsets independently. Utilizing Linux kernel zero-copy (`sendfile` bypassing user-space memory buffers) and batch compression, Kafka handles millions of events/sec at multi-gigabyte throughput. De facto backbone for event-driven streaming and audit logging.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Kafka Offset Tabanlı Tüketim Modeli Simülasyonu
import java.util.*;

public class KafkaCommitLogSim {
    private final List<String> commitLog = new ArrayList<>();

    public synchronized long appendEvent(String event) {
        commitLog.add(event);
        return commitLog.size() - 1; // Üretilen Offset değeri
    }

    // Tüketici kendi offset'i ile istediği yerden okur (Replay yeteneği!)
    public synchronized List<String> pollEvents(long fromOffset, int batchSize) {
        List<String> results = new ArrayList<>();
        for (long i = fromOffset; i < commitLog.size() && results.size() < batchSize; i++) {
            results.add(commitLog.get((int) i));
        }
        return results;
    }
}

void main() {
    var kafka = new KafkaCommitLogSim();
    kafka.appendEvent("ORDER_CREATED_#1");
    kafka.appendEvent("ORDER_PAID_#1");
    kafka.appendEvent("ORDER_SHIPPED_#1");

    // Tüketici A 0. offset'ten okuyor:
    IO.println("Consumer A (Tüm Tarihçe): " + kafka.pollEvents(0, 10));
    // Tüketici B sadece 2. offset'ten sonrasını okuyor:
    IO.println("Consumer B (Sadece Kargo): " + kafka.pollEvents(2, 10));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Kafka'yı basit bir e-posta gönderme kuyruğu için kullanırım' demeyin! Küçük ölçekli, basit iş kuyruklarında Kafka'nın partition ve consumer group karmaşıklığı aşırı mühendisliktir (over-engineering); basit kuyruklar için RabbitMQ veya AWS SQS daha uygundur.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not recommend Kafka for trivial background email dispatch queues. Operating Kafka clusters incurs ZooKeeper/KRaft and partition management complexity. For simple task dispatching, propose RabbitMQ or AWS SQS.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- RabbitMQ mesajı işlendikten sonra siler; akıllı yönlendirme ve iş kuyrukları için idealdir.
- Kafka mesajları kütükte (log) saklar; geriye dönük tekrar oynatma (replayability) sunar.
- Zero-copy disk transferi sayesinde Kafka saniyede milyonlarca olayı minimum CPU ile taşır.

🎯 Key Takeaways for the Interview

- RabbitMQ expunges messages upon delivery; ideal for complex routing and job queues.
- Kafka stores immutable commit logs, allowing consumers to rewind and reprocess history.
- Zero-copy OS primitives allow Kafka to stream millions of events/sec at bare-metal speeds.

27

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Apache Kafka İç Mimarisi: Topics, Partitions, Consumer Groups ve Segmentler

Apache Kafka Internals: Topics, Partitions, Consumer Groups & Segments

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ KAFKA TOPIC: "orders" (3 PARTITION'A BÖLÜNMÜŞ) ]
Partition 0: [0][1][2][3][4] ──► Consumer 1 (Pod A)
Partition 1: [0][1][2][3]    ──► Consumer 2 (Pod B)   Consumer Group: "Billing-Service"
Partition 2: [0][1][2]       ──► Consumer 3 (Pod C)
(Her partition yalnızca TEK bir consumer tarafından okunur -> Paralellik & Sıralılık!)

[ PARTITION DISK YAPISI: .log ve .index DOSYALARI ]
Segment 000000.log  (1 GB Dolunca kapanır) ──► Segment 000001.log (Yeni aktif kütük)
Segment 000000.index (Seyrek İndeks - Sparse Index: Binary Search ile O(log N) arama)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kafka Topic'ini 3 şeritli bir otoyol (3 Partition) olarak düşünün. Eğer tek şerit olsaydı tüm arabalar tek sıra halinde giderdi ve trafik tıkanırdı. 3 şerit açarak trafiği 3 katına çıkarırsınız! Şeritlerin çıkışına da 3 kişilik bir gişe görevlisi ekibi (Consumer Group) koyarsınız: Her görevli yalnızca KENDİ şeridini izler. Arabalar kendi şeritlerinde asla birbirini sollayamaz (Şerit içinde kesin sıra garantisi!); ancak şeritler birbirini beklemez!

🇬🇧 ENGLISH (ELI5)

Think of a Kafka Topic as a 3-lane highway (3 Partitions). A single-lane road bottlenecks all cars into a single file crawl. Adding 3 lanes triples traffic throughput! At the highway exit, assign 3 toll clerks belonging to the same union (Consumer Group): each clerk exclusively monitors their dedicated lane. Within a single lane, cars cannot overtake one another (strict in-order sequence within a partition!); but different lanes progress in parallel!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kafka mimarisinin temel yapı taşları: 1) Topic: Mantıksal veri akışı kategorisi. 2) Partition: Yatay ölçekleme ve paralellik birimidir. Mesajlar anahtarın (Key) hash değerine göre (`MurmurHash2(key) % partition_count`) ilgili bölüme yazılır. Aynı anahtara sahip tüm mesajlar KESİNLİKLE aynı partition'a gider ve sıralı işlenir. 3) Consumer Group: Bir grup içindeki tüketici sayısı partition sayısını aşamaz; her partition o gruptaki yalnızca TEK bir tüketiciye atanır. 4) Disk Segmentleri: Her partition diskte 1 GB'lık segment dosyalarına bölünür (`.log` ve seyrek indeks `.index`). İşletim sistemi sayfa önbelleği (PageCache) kullanılarak disk aramaları RAM hızına çıkarılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Kafka internal architecture: 1) Partitioning: The fundamental unit of scalability. Keyed messages route via `MurmurHash2(key) % num_partitions`, guaranteeing deterministic partition assignment and strict ordering for identical keys (e.g. all events for `user_101`). 2) Consumer Groups: Workload parallelization engine. A partition is consumed by at most one worker per consumer group; exceeding worker counts relative to partition counts leaves surplus consumers idle. 3) Storage Segments: Partitions split on disk into bounded segment files (typically 1GB `.log` alongside `.index` sparse indexes). Kafka relies heavily on the Linux OS PageCache rather than JVM heap memory, avoiding garbage collection pauses while maximizing sequential disk IO.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Kafka Partition Yönlendirme ve Consumer Group Eşleme Simülasyonu
import java.util.*;

public class KafkaPartitionRouter {
    private final int partitionCount;

    public KafkaPartitionRouter(int partitionCount) {
        this.partitionCount = partitionCount;
    }

    public int assignPartition(String messageKey) {
        if (messageKey == null) {
            return ThreadLocalRandom.current().nextInt(partitionCount); // Round-Robin
        }
        // Aynı anahtar DAİMA aynı partition'a gider!
        return Math.abs(messageKey.hashCode() % partitionCount);
    }
}

void main() {
    var router = new KafkaPartitionRouter(3); // 3 Partition'lı Topic
    IO.println("User 'A' Olayı -> Partition #" + router.assignPartition("user:A"));
    IO.println("User 'B' Olayı -> Partition #" + router.assignPartition("user:B"));
    IO.println("User 'A' İkinci Olayı -> Partition #" + router.assignPartition("user:A") + " (Sıra
Korundu!)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Tüm sistem için global bir mesaj sırası garanti ederim' derseniz elenirsiniz! Kafka'da mesaj sırası GLOBAL DEĞİLDİR; sadece AYNI PARTITION içinde sıralıdır. Global sıra istiyorsanız Topic'i 1 Partition'a düşürmelisiniz; bu da yatay ölçeklemeyi yok eder.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never claim Kafka guarantees global ordering across an entire topic. Kafka strictly guarantees ordering ONLY WITHIN A SINGLE PARTITION. Global ordering requires restricting a topic to 1 single partition, which demolishes horizontal throughput.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Kafka'da paralellik sınırı partition sayısı kadardır.
- Aynı varlığa ait mesajlar aynı Key ile gönderilerek sıralı işleme garantilenir.
- Kafka JVM heap yerine işletim sistemi PageCache'ini kullanarak GC duraklamalarını sıfırlar.

🎯 Key Takeaways for the Interview

- The partition count establishes the hard ceiling on consumer parallelism per consumer group.
- Passing consistent message keys guarantees partition-level ordering for state machines.
- Kafka relies on OS PageCache rather than JVM heap, neutralizing GC pauses entirely.

28

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Mesaj Teslimat Garantileri: At-Most-Once, At-Least-Once ve Exactly-Once (EOS)

Message Delivery Guarantees: At-Most-Once, At-Least-Once & EOS

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. AT-MOST-ONCE (En Fazla Bir Kez) ]
Mesajı al ──► Hemen ACK at ──► Mesajı İşle (Sunucu ÇÖKERSE -> MESAJ KAYBOLUR!)

[ 2. AT-LEAST-ONCE (En Az Bir Kez - Endüstri Standardı!) ]
Mesajı al ──► Mesajı İşle ──► DB'ye Yaz ──► ACK at (ACK ağda kaybolursa -> MÜKERRER İŞLENİR!)
(Çözüm: Tüketiciyi IDEMPOTENT tasarlamak!)

[ 3. EXACTLY-ONCE SEMANTICS (EOS - Tam Olarak Bir Kez) ]
İki Fazlı Onay (2PC) + Kafka Transaksiyonel Producer + Read-Committed İzolasyonu.
(En güvenli ancak en maliyetli ve karmaşık modeldir)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Birine mektup göndermenin 3 yolu: 1) At-Most-Once (Aptal Postacı): Mektubu posta kutusuna atar ve hemen arkasını dönüp gider. Mektup yağmurda çürürse çürür; asla tekrar denemez (Kayıp olabilir). 2) At-Least-Once (İnatçı Postacı): Kapıyı çalar, mektubu verir. Ancak siz 'Aldım' demeden önce gök gürültüsünden sesinizi duymazsa ertesi gün aynı mektuptan bir tane daha getirir! Mektup asla kaybolmaz ama iki kez teslim edilebilir (Mükerrerlik). 3) Exactly-Once (Noter Tasdikli Postacı): İki tarafın da imza attığı, noter kütüğüne işlenen kusursuz sistemdir; mektup ne kaybolur ne de iki kez gelir!

🇬🇧 ENGLISH (ELI5)

Three ways to deliver a package: 1) At-Most-Once: Drop the parcel on the porch and run. If a porch pirate steals it, tough luck; nobody redelivers (messages can be lost). 2) At-Least-Once: Ring the bell and hand over the parcel. If lightning strikes and the courier doesn't hear your 'Thank you' receipt, they return tomorrow with a duplicate parcel! Nothing is ever lost, but duplicates happen. 3) Exactly-Once: A bonded courier and biometric notary signing the delivery ledger synchronously: the package is delivered exactly one time with zero loss and zero duplicates!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) At-Most-Once: Tüketici mesajı okur okumaz offset'i commit eder, sonra işler. İşleme sırasında çökerse mesaj kaybolur (Sıfır veri kaybı toleransı olan sistemlerde kabul edilemez). 2) At-Least-Once: Tüketici önce mesajı işler, veritabanına kaydeder, en son offset commit eder. Commit sırasında ağ koparsa mesaj tekrar tüketilir (Mükerrerlik riski). Bu model endüstri standardıdır ve tüketici katmanında Idempotency (tekilleştirme) ile çözülür. 3) Exactly-Once Semantics (EOS): Kafka'da `enable.idempotence=true` (PID + Sequence Number) ve `TransactionalId` ile iki fazlı onay (2PC) kullanılarak sağlanır. Kafka'dan Kafka'ya (Stream-to-Stream) akışlarda kusursuzdur; ancak harici bir veritabanına yazarken harici sistemin de 2PC desteklemesi gerekir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) At-Most-Once: Consumers commit offset before processing. If processing crashes, the message is lost forever (unacceptable for financial systems). 2) At-Least-Once: Consumers process payload and persist to DB before committing offsets. If the network drops prior to offset commit, the broker redelivers on restart. This is the enterprise industry standard, pairing at-least-once delivery with idempotent downstream consumers. 3) Exactly-Once Semantics (EOS): Enforced in Kafka via idempotent producers (Producer ID + monotonic Sequence Numbers) and atomic transactions across multiple topic partitions using a Two-Phase Commit transaction coordinator. While native for Kafka-to-Kafka stream pipelines, sinking to third-party databases requires distributed transactions (XA) or idempotent UPSERTs.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Idempotent Tüketici Deseni (At-Least-Once Güvenliği)
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class IdempotentConsumer {
    // İşlenmiş benzersiz mesaj ID'leri tablosu (Redis veya DB Unique Constraint)
    private final Set<String> processedMessageIds = ConcurrentHashMap.newKeySet();

    public record Message(String messageId, String payload) {}

    public synchronized boolean processMessage(Message msg) {
        // Eğer bu mesaj ID'si daha önce başarıyla işlendiyse: PAS GEÇ!
        if (!processedMessageIds.add(msg.messageId())) {
            IO.println("⚠ MÜKERRER MESAJ TESPİT EDİLDİ! Pas geçiliyor: " + msg.messageId());
            return false;
        }

        // İlk kez işleniyor:
        IO.println("✔ Mesaj başarıyla işlendi: " + msg.payload());
        return true;
    }
}

void main() {
    var consumer = new IdempotentConsumer();
    var msg = new IdempotentConsumer.Message("msg_uuid_9981", "Bakiye Düş: 50 TL");

    consumer.processMessage(msg); // 1. Teslimat -> İşlenir
    consumer.processMessage(msg); // 2. Ağ Yeniden Denemesi (Retry) -> Tekilleştirilir!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bizim sistemimiz uçtan uca Exactly-Once çalışıyor' iddiasında bulunurken dikkatli olun! Dış dünyadaki bir SMS veya e-posta servisine mesaj gönderirken Exactly-Once fiziksel olarak imkansızdır (SMS atıldıktan sonra elektrik kesilirse SMS iki kez gidebilir). Mülakatçıya gerçek dünyada çözümün 'At-Least-Once + Idempotency' olduğunu söyleyin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Be wary of blindly claiming end-to-end Exactly-Once across heterogeneous systems. Sending an SMS or email cannot be rolled back via two-phase commit. Reassure the interviewer that real-world production relies on At-Least-Once paired with Idempotent consumers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- At-Least-Once teslimat, mesaj kayıplarını sıfırlamak için en güvenilir temel modeldir.
- Mükerrer teslimat riskini bertaraf etmenin anahtarı tüketiciyi Idempotent yapmaktır.
- Kafka EOS, akış işleme (Kafka Streams) içinde tam olarak bir kez işlemeyi garanti eder.

🎯 Key Takeaways for the Interview

- At-Least-Once is the pragmatic distributed foundation to guarantee zero data loss.
- Pairing At-Least-Once with idempotent consumer logic neutralizes duplicate deliveries.
- Kafka transactional APIs guarantee Exactly-Once within internal stream processing topologies.

29

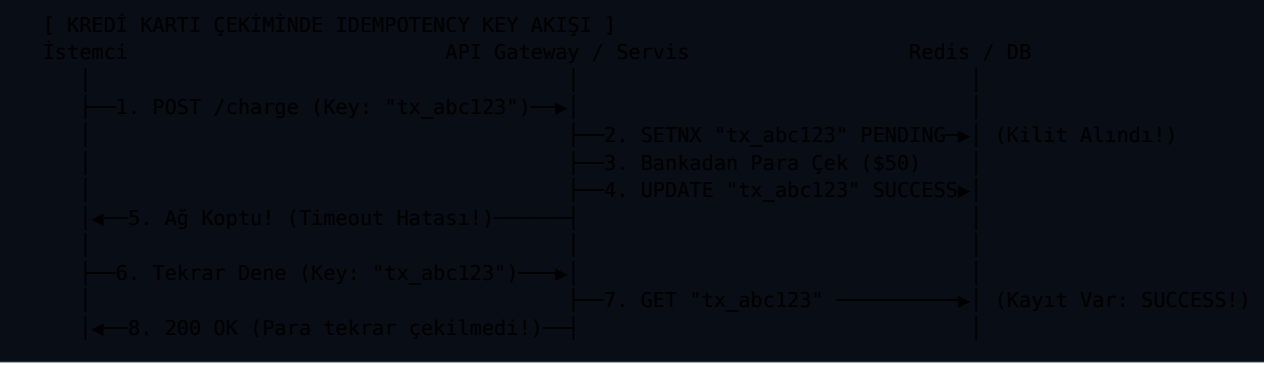
BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Idempotency (Tekrarlanabilirlik): Ağ Yeniden Denemelerinde Müker-rerliği Önleme

Idempotency Patterns: Preventing Double Charges & Duplicates

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Asansör çağrı düğmesini düşünün: Düğmeye bir kez basarsanız ışığı yanar ve asansör kata doğru hareket eder. Sabırsızlanıp düğmeye 10 kez daha basarsanız ne olur? Asansör 10 kat daha hızlı gelmez, 10 tane ayrı asansör de çağrılmaz! Düğmeye 1 kez basmakla 100 kez basmak aynı sonucu üretir. İşte matematikte ve yazılımda buna `Idempotent` (Tekrarlanabilir) denir: Kullanıcı interneti koptu diye 'Öde' butonuna 5 kez tıklasa bile kredi kartından sadece 1 kez 50 TL çekil-melidir!

🇬🇧 ENGLISH (ELI5)

Think of an elevator call button: Pressing it once illuminates the light and dispatches the cab. If you impatiently mash the button 10 more times, does it arrive 10 times faster? Does it spawn 10 separate elevators? No! Pressing it once or 100 times yields the exact same state. That is `Idempotency`. If a customer's phone switches from Wi-Fi to 5G during checkout and re-submits a payment 5 times, their credit card must be charged exactly once!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık sistemlerde ağ zaman aşımları (Timeout) nedeniyle istemciler istekleri yeniden dener (Retry). Bir POST isteği doğası gereği idempotent değildir. Çözüm: 1) İstemci Tarafında Benzersiz Anahtar (Idempotency Key): İstemci isteği göndermeden önce bir UUID üretir (`Idempotency-Key: 9b1deb4d...`) ve HTTP başlığına ekler. 2) Dağıtık Kilit & Durum Tablosu (Redis / DB): API Gateway anahtarı alır; `SET key PENDING NX EX 120` ile atomik kilit oluşturur. İşlem tamamlandığında sonuç (Response Body) bu anahtarın altına kaydedilir. 3) Mükerrer İstek Geldiğinde: Eğer anahtar `SUCCESS` durumundaysa, işlem tekrar çalıştırılmaz; önbellekteki ilk başarılı yanıt aynen geri dönülür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

In distributed microservices, network timeouts inevitably trigger client retries. While GET and PUT are inherently idempotent, HTTP POST mutations are not. Enterprise Idempotency Pattern: 1) Client Idempotency Key: The client generates a unique UUID (`Idempotency-Key: 9b1deb4d...`) passed in headers. 2) Atomic State Locking: The receiver executes an atomic lock (`SET key PENDING NX EX 60`) in Redis or an RDBMS unique table constraint. 3) Response Caching: When the transactional operation succeeds, the final response payload is persisted against the idempotency key. Subsequent retry attempts detect the existing key and return the cached receipt immediately without re-executing side effects.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Idempotency Key Motoru (Ödeme Güvenlik Kalkanı)
import java.util.concurrent.*;

public class PaymentIdempotencyEngine {
    public enum TxStatus { PENDING, COMPLETED }
    public record TxRecord(TxStatus status, String responsePayload) {}

    private final ConcurrentMap<String, TxRecord> store = new ConcurrentHashMap<>();

    public synchronized String processPayment(String idempotencyKey, double amount) {
        TxRecord existing = store.get(idempotencyKey);
        if (existing != null) {
            if (existing.status() == TxStatus.COMPLETED) {
                IO.println("⚡ Önbellekteki Makbuz Döndü: " + existing.responsePayload());
                return existing.responsePayload();
            }
            throw new IllegalStateException("İşlem şu anda başka bir thread tarafından yürütülüyor!");
        }

        // 1. Durumu PENDING yap
        store.put(idempotencyKey, new TxRecord(TxStatus.PENDING, null));

        // 2. Banka POS simülasyonu
        IO.println(">>> Bankadan $" + amount + " çekildi! (Kart İşlemi Yapıldı)");
        String receipt = "RECEIPT_TX_" + System.currentTimeMillis();

        // 3. Durumu COMPLETED yap ve sonucu sakla
        store.put(idempotencyKey, new TxRecord(TxStatus.COMPLETED, receipt));
        return receipt;
    }
}

void main() {
    var engine = new PaymentIdempotencyEngine();
    String key = "req_uuid_550e8400";

    // 1. İstek (Başarılı)
    engine.processPayment(key, 100.0);
    // 2. Ağ hatası nedeniyle aynı isteğin tekrarı (Tekrar para çekilmez!)
    engine.processPayment(key, 100.0);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Idempotency anahtarlarını veritabanında sonsuza kadar saklamayın! Bu tablolar milyonlarca gereksiz anahtarla şişer. Idempotency anahtarlarına 24 veya 48 saatlik bir TTL (Time-To-Live) belirleyin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not persist idempotency keys indefinitely. Storing transient keys permanently bloats storage. Enforce a rolling 24-to-48 hour TTL window, which easily exceeds maximum client retry horizons.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Idempotency Key, ağ hatalarında mükerrer para çekilmesini ve kayıt açılmasını engeller.

• HTTP POST istekleri istemcinin ürettiği benzersiz bir token ile idempotent kılınır.

• Stripe, PayPal ve Adyen gibi ödeme devlerinin tüm API'leri bu desenle korunur.

🎯 Key Takeaways for the Interview

• Idempotency keys eliminate double-billing and duplicate resource creation during retries.

• Pairing client-generated UUID tokens with atomic server-side state locks ensures idempotency.

• Industry payment APIs (Stripe, Adyen) mandate Idempotency-Key headers on all mutations.

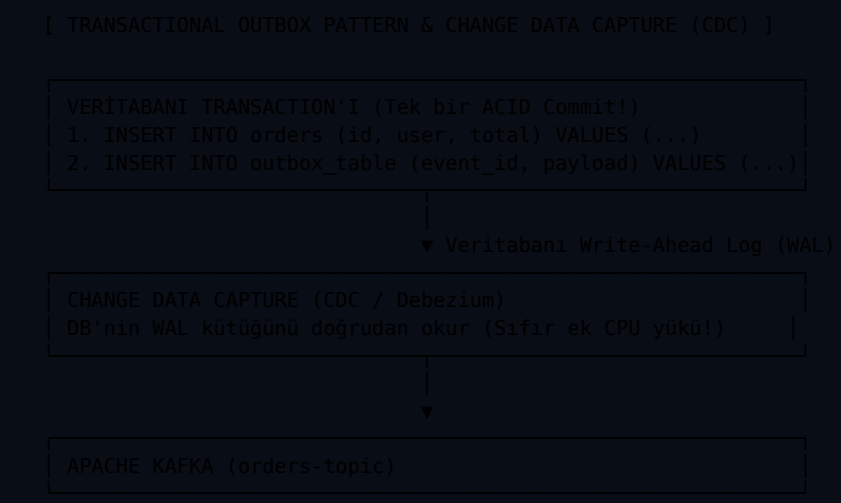
30

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Transactional Outbox Pattern ve Change Data Capture (CDC / Debezium)

Transactional Outbox Pattern & Change Data Capture (CDC)

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette sözleşme imzaladığınızı düşünün. Normalde yapılması gereken iki iş vardır: 1) Sözleşmeyi şirket kasasına kilitlemek (DB kaydı), 2) Müşteriye kargo göndermek (Kafka olayı). Eğer sözleşmeyi kasaya koyduktan hemen sonra elektrikler kesilirse ve kargocuya haber veremezseniz sistem bozulur! Transactional Outbox deseni şudur: Kasanın içine küçük bir 'Giden Evrak Kutusu' (Outbox Tablosu) koyarsınız. Sözleşmeyi kilitlerken kargo emrini de AYNI kasanın içine tek bir anahtar çevirmeyle kilitlersiniz (ACID Transaction)! Sonra kargo çırağı (Debezium/CDC) kasadaki kutuyu okur ve kargoyu güvenle yola çıkarır!

🇬🇧 ENGLISH (ELI5)

Imagine signing a commercial contract requiring two atomic actions: 1) Lock the contract in the vault (Database write), 2) Dispatch a courier to the client (Kafka event). If lightning strikes right after locking the vault, the courier is never alerted, corrupting the system! The Transactional Outbox Pattern solves this: Place an 'Outgoing Mailbox' inside the exact same vault! In a single twist of the vault key (one atomic ACID transaction), you lock both the contract and the courier dispatch ticket together! A dedicated courier bot (Debezium / CDC) monitors the outgoing mailbox and dispatches events with zero risk of desynchronization!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

İki Fazlı İki Ayrı Sistem Problemi (Dual-Write Problem): Uygulama önce veritabanına `INSERT` yapıp ardından Kafka'ya `send()` yaparsa; DB başarılı olup Kafka çağrısı çökerse sistem tutarsızlaşır. Transactional Outbox Deseni bu ikili yazma açığını çözer: 1) İş mantığı verisi (`orders`) ve dışarıya fırlatılacak olay (`outbox`) AYNI veritabanında TEK bir yerel ACID transaction içinde commit edilir. 2) Change Data Capture (CDC / Debezium): Veritabanının işlem günlüğünü (PostgreSQL WAL veya MySQL Binlog) doğrudan dinler. Veritabanına hiçbir sorgu atmadan, disk seviyesindeki binary log değişikliklerini yakalayarak Kafka'ya sıfır veri kaybıyla yazar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Dual-Write Anti-Pattern: A service writes to a relational database and then publishes to Kafka. If the database commits but the network blip crashes the Kafka producer, data desynchronizes permanently. The Transactional Outbox Pattern cures this: 1) Business entities and an `outbox` event table commit together in a single local ACID database transaction. 2) Change Data Capture (CDC / Debezium): An external tailer intercepts the database Write-Ahead Log (WAL / Binlog) directly at the storage engine level with zero query polling overhead, streaming mutations into Kafka with absolute At-Least-Once delivery guarantees.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Transactional Outbox Veri Yapısı ve Atomic Commit Temsili
public class TransactionalOutboxPattern {
    public record OrderRecord(long orderId, String customer, double amount) {}
    public record OutboxMessage(String eventId, String aggregateType, String payload) {}

    // Tek bir yerel ACID veritabanı transaction'ında çalışan metod:
    public void createOrderWithOutbox(OrderRecord order) {
        // BEGIN TRANSACTION;

        // 1. İş tablosuna yaz:
        IO.println("1. DB INSERT INTO orders: ID=" + order.orderId());

        // 2. Outbox tablosuna yaz (Aynı DB Transaction'ı!):
        var outboxEvent = new OutboxMessage(
            "evt_" + System.currentTimeMillis(),
            "ORDER",
            "{"orderId": " + order.orderId() + ", "amount": " + order.amount() + "}"
        );
        IO.println("2. DB INSERT INTO outbox: EventID=" + outboxEvent.eventId());

        // COMMIT TRANSACTION;
        // CDC (Debezium) arka planda WAL'ı okuyup Kafka'ya iletir!
    }
}

void main() {
    var outbox = new TransactionalOutboxPattern();
    outbox.createOrderWithOutbox(new TransactionalOutboxPattern.OrderRecord(9021L, "Ali Bozkurt",
250.0));
    IO.println("✔ Atomik Outbox Kaydı Tamamlandı (Dual-Write Engellendi)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Outbox tablosunu periyodik olarak `SELECT * FROM outbox WHERE sent = false` şeklinde polling yaparak okumak mülakatta Junior cevabıdır! Tablo büyüdükçe veritabanını kilitler. Senior/Staff seviye cevap: Debezium gibi bir CDC aracının veritabanının binary Write-Ahead Log'unu (WAL) doğrudan dinlemesidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Proposing a background cron poller (`SELECT * FROM outbox WHERE processed = false`) is an interview red flag that degrades database CPU. The Staff-level answer is transaction log tailing via Change Data Capture (CDC with Debezium reading PostgreSQL WAL).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Dual-Write problemi, DB ve Kafka'ya aynı anda ayrı ayrı yazmaya çalışmanın felaketidir.
- Outbox tablosu, olayları iş verisiyle aynı ACID transaction'ında güvenceye alır.
- Change Data Capture (CDC / Debezium) veritabanı WAL kütüğünü dinleyerek sıfır yükle çalışır.

🎯 Key Takeaways for the Interview

- The Dual-Write anti-pattern risks fatal desynchronization between DB state and event topics.
- The Outbox table commits domain entities and event envelopes within a single ACID transaction.
- CDC engines (Debezium) tail database WAL logs with zero query polling penalty.